

Faculdade de Ciências da Universidade do Porto



Segurança de Sistemas e Dados

Turma 1 Sem grupo

Napoleão Martins

Índice

Introdução	3
Contextualização	3
Objetivos do projeto	3
Estrutura do Relatório	3
Tecnologias utilizadas	3
Backend	3
Criptografia	4
Frontend	4
Orquestração e Infraestrutura	4
Arquitetura de redes e nós	4
Topologia de Rede	4
Controlo de acesso baseado em Identidade	5
Implementação da Blockchain	5
Estrutura do bloco e transação	5
Mecanismo de consenso	6
Segurança e Validação	6
Lógica de Negócio dos leilões	7
Injeção dinâmica de estados	7
Mecanismo de fecho	7
Interface do utilizador	7
Dashboard Dinâmico	7
Lógica condicional	7
Testes e Resiliência	8
Automação do Setup	8
Auditoria e Logs	8
Conclusão	8
Referências	9

Introdução

Contextualização

O Blockchain é um livro de atribuição da razão, disponível para o compartilhamento e sem possibilidade de mutar. Permite a possibilidade de registo de transações e monitoramento de alvos ativos numa rede empresarial, dando a possibilidade de uma única fonte credível.

A principal vantagem da sua utilização é a sua capacidade de proporcionar segurança sem depender de identidades intermediárias, como bancos, mantendo a segurança e a transparência.

No caso de um sistema de leilões, para além disso, ainda garante transparência, imutabilidade e execução automática de funções através de contratos desenvolvidos, impedindo fraudes.

Objetivos do projeto

O principal objetivo do projeto foi construir uma blockchain de raiz robusta, fiável e interativa, em Rust, de forma a simular um leilão e todas as suas possibilidades, como iniciar, fechar e licitar, numa rede p2p (*Peer-to-Peer*), dinâmica e segura.

Estrutura do Relatório

Nas secções seguintes, irei pormenorizar quais as tecnologias que usei, a arquitetura de rede, a implementação do blockchain, a lógica de negócio dos leilões, a interface do utilizador, os testes e resiliência, uma conclusão e finalmente as referências que utilizei para desenvolver este relatório.

Tecnologias utilizadas

Backend

Para realizar o backend do programa, eu usei 3 tecnologias, nomeadamente:

- Rust, devido à sua alta performance e segurança da memória.
- Axum, framework utilizada para API REST, foi-me muito útil para mapear os botões do dashboard para as funções do Rust e para a injeção da blockchain dentro das minhas funções.
- Tokio é um runtime assíncrono. Como a blockchain precisa de estar sempre pronta para receber lances dos leilões e minerar blocos, o Rust usa o Tokio para gerir essas mesmas tarefas no fundo.

Criptografia

Usei dois mecanismos criptográficos:

- SHA-256, sendo uma função de hashing, usada para encadear blocos e garantir que o seu histórico é imutável.

Também usada para fazer a mineração dos leilões, uma vez que o meu Proof of work usamos este mecanismo num loop.

- Ed25519 (Dalek) que é um sistema de criptografia de chave assimétrica focado em assinaturas digitais. Eu usei para criar as transações, mais especificamente, quando um comprador faz um bid ou o vendedor cria o leilão, esse nó assina essa transação usando a chave privada.

Frontend

Para criar a interface com o utilizador usei HTML, CSS e JavaScript, linguagens de modelação e desenho de páginas web estáticas, o seu formato e aspeto visual e a possibilidade de existência de avisos. Todos estes comunicam diretamente com o API REST de forma a realizar todas os requisitos que quero do meu leilão. Essa comunicação é feita através de pedidos http assíncronos, utilizando uma API do JS chamada fetch.

Orquestração e Infraestrutura

Usei o Docker e o Docker compose para simular uma rede de computadores independentes. De forma a automatizar o envio de comandos e o arranque do programa, criei um ficheiro, que ao ser executado na Shell do projeto, já faz a limpeza de volumes antigos, faz um monitoramento ativo do arranque, onde é verificada a prontidão do api rust, se houver mais de x tentativas, o programa não arranca e conseguimos perceber que existe um erro na compilação. Após essa verificação, decidi atribuir uma injeção inicial de 3 leilões com as suas características, assim como a respetiva mineração. Após tudo estar em conformidade, será dada a mensagem do sucesso e respetiva disponibilidade para correr a interface com o utilizador.

Arquitetura de redes e nós

Topologia de Rede

Comecei por isolar os nós autónomos. No meu ficheiro docker-compose.yml, cada nó da rede foi declarado como serviço independente, ou seja, cada contentor tem a sua própria instância compilada e cada contentor funciona como se fosse independente, em termos de sistema de ficheiros e memória.

Depois fiz um mapeamento de portas no Docker, onde cada contentor do ecossistema faz com que o servidor via Axum esteja configurado para escutar sempre a porta interna. Atribuí a porta 9080 para mapear para o nó Bootstrap, a porta 9081 para mapear para o nó Vendedor, a porta 9082 para mapear para o Comprador 1, a porta 9083 para mapear para o Comprador 2 e a porta 9084 para mapear para o Comprador 3.

Finalmente tornei a rede virtual mais dinâmica, onde além de dispor as portas para o exterior, o Docker compõe e cria instantaneamente uma vpn interna. Isto permite que eles comuniquem entre si nos bastidores usando o nome em vez do ip.

Controlo de acesso baseado em Identidade

Existem 3 tipos de nós onde cada um tem tarefas e autorizações diferentes como:

- O nó Bootstrap mantém a estabilidade da rede onde este pode minerar e testar o blockchain, garantindo o bom funcionamento do sistema.
- O nó Vendedor, sendo o único responsável por criar leilões e encerrar os mesmos.
- Os nós Compradores, que são os únicos com a capacidade de submeter aumentos de valor ou lances, os bids.

Implementação da Blockchain

Estrutura do bloco e transação

Para que a blockchain funcione sem falhas e de forma segura, cada função tem uma lógica bem definida, como:

- Os campos de transação, onde cada transação representa uma ação no mercado, como criar ou fechar um leilão. Neste campo, o remetente identifica quem está a realizar a ação mapeia o nó e a chave, obrigando saber quem é o dono do produto. Depois o destinatário é o alvo da transação, onde este campo guarda os prefixos de controlo, respetivamente, o `auction_start:Nome_do_Produto` usado para criar o item do `ID_do_Leilão` usado quando um comprador faz um lance e o `close_auction:ID_do_Leilão` usado quando o vendedor encerra o item. O montante representa o valor da ação, o nonce serve para evitar ataques de replay e a assinatura serve para que o vetor de bytes seja gerado pela criptografia de Ed25519.
- Os campos de um bloco, que é a caixa de segurança que sela um conjunto de transações pendentes. O seu índice serve para manter uma ordenação

cronológica, o timestamp regista o momento exato em que o bloco foi minerado, ajudando a auditar quando os leilões foram consolidados e o hash anterior que serve para guardar o código criptográfico.

Mecanismo de consenso

De forma a realizar um consenso do Proof of Work com a prova de trabalho do sistema, utilizei a função `mine_block`, que funciona como um selo de segurança que consolida transações. É feito um agrupamento de dados, onde a função ao ser invocada, recolhe todas as transações em espera e junta-as aos dados do novo bloco. Depois é realizado um loop criptográfico, onde a função inicia um contador `nonce`, concatenando todos os dados junto do bloco e calcula o hash resultante utilizando o sha-256.

De seguida uma verificação da dificuldade, onde após o cálculo, a função converte o hash numa string e verifica se o seu início corresponde aos critérios de dificuldade predefinidos.

Finalmente é feita uma resolução e registo onde o hash tem de cumprir um determinado requisito, se não cumprir, o `nonce` é incrementado e o ciclo repete-se, se o hash cumprir o desafio criptográfico é resolvido e o loop interrompido. O `nonce` vencedor e o hash final são gravados e este é adicionado à blockchain, limpando-se da lista das transações pendentes.

Segurança e Validação

Como requisitos de segurança e validação, são feitos dois mecanismos:

- Geração dinâmica de chaves, onde cada nó, ao ser inicializado pelo Docker, gera dinamicamente na sua memória um par de chaves com o Ed25519, uma chave privada, que fica guardada em segredo absoluto no nó e serve para assinar as transações, e uma pública que é partilhada abertamente com a rede e serve como identidade pública.
- A função `is_signature_valid` que impede falsificações, onde, quando é feito um lance num nó, é criado um bloco e encriptado um hash desses dados com a chave privada, obtendo a assinatura.

Se um utilizador mal-intencionado tentar falsificar, a função entra em ação antes do rust aceitar, pega nos dados, na chave pública e na assinatura anexada e o algoritmo faz a validação, se o conteúdo tiver sido alterado ou se a assinatura não estiver a ser gerada pela chave privada real do respetivo nó, a validação falha.

Lógica de Negócio dos leilões

Injeção dinâmica de estados

No modelo tradicional, os dados ficam em tabelas estáticas da base de dados relacional. Com o meu blockchain, não existe nenhuma tabela de leilões. Os leilões são calculados dinamicamente em tempo real através da reconstrução da cadeia. Isso é feito em 3 fases, nomeadamente o estado dinâmico, usado para saber quais os leilões ativos, onde é feita uma leitura sequencial, seguido de uma descoberta de produtos, onde o código encontra a transação cujo destinatário tem o prefixo `auction_start:Nome_do_Produto`, ele sabe que um novo leilão foi inaugurado, e finalmente o cálculo do `current_bid`, onde o sistema continua a iterar. Sempre que encontrar uma transação, compara valores e guarda o montante mais alto.

Mecanismo de fecho

Quando um vendedor decide encerrar o leilão, o sistema em vez de apagar, injeta uma nova transação com prefixo `close_auction:id_do_leilao`. É feita em três fases, nomeadamente o registo permanente, onde a transação é assinada e guardada, um filtro em tempo real, quando a interface web solicita a lista de leilões, e finalmente a exclusão do mercado, onde se o código encontrar o marcador do `close`, ele remove ou ignora esse leilão na lista que aparece no ecrã.

Interface do utilizador

Dashboard Dinâmico

A alteração do nó baseia-se na manipulação dinâmica do DOM (Document Object Model) e de uma variação do JavaScript, sendo feito em 3 passos: é escutado o evento, onde no html, existe um dropdown que tem um ID, ficando á escuta de qualquer alteração feita com esse; é feita uma atualização da porta, nomeadamente quando escolhemos um nó, o JS apanha o valor associado a essa opção e atualiza a variável do URL; e finalmente é feita uma modificação da interface, onde o código limpa os dados antigos e atualiza o mercado numa função. Essa função faz um novo fetch, alterando os títulos e as tabelas para aquele nó só ver o que é autorizado.

Lógica condicional

A visibilidade condicional baseia-se na manipulação de propriedades do CSS, através do JS, sendo ocorrido da seguinte forma: é feita uma deteção de qual perfil ou nó é selecionado no dropdown, sendo o JS responsável por isso; depois é feita

uma validação , onde o código vê se esse texto contém uma palavra-chave; e finalmente é feito um controlo de visibilidade com os botões de terminar o leilão, onde o JavaScript faz um varrimento de todos os botões vermelhos e altera o estilo se for selecionado um leilão que não possa fazer determinada tarefa, e a ligação da página criar.html, aplicando o mesmo princípio, garantindo que só determinado nó pode fazer essa tarefa.

Testes e Resiliência

Automação do Setup

O meu `iniciar_demonstracao.ps1` é capaz de eliminar resíduos criptográficos, com a flag `-v`, destruindo completamente os nós e valores antigos. Isto é muito importante porque, caso essa limpeza não fosse efetuada, causaria erros de sincronização e de memória. A flag `--remove-orphans` limpa outros dados indesejados em segundo plano, garantindo que o programa está completamente limpo e vazio. O programa também garante que o estado está sempre limpo, ao forçar o encerramento e limpeza, assegurando que o p2p inicia sempre do zero, permitindo a visualização do crescimento da cadeia, sem poluição. Finalmente, e como exemplo, é feita uma injeção automática de dados, ajudando assim o utilizador com 3 exemplos de leilões de teste, fazendo subir a infraestrutura e a possibilidade de injetar dados, deixando os nós comunicar com as respetivas portas.

Auditoria e Logs

Na auditoria de integridade usei o botão “Test Block” que serve como uma ferramenta de auditoria instantânea da integridade do sistema. É feito um pedido à API, que verifica se o `previous_hash` coincide com o hash real. A interface devolve uma mensagem de confirmação com o tamanho dessa cadeia e o último hash guardado, provando que o blockchain está funcional e nenhum dado foi corrompido. Também faço uma validação visual, onde o terminal exibe em tempo real o comportamento do Proof-of-Work. Quando clicamos em minerar, o ecrã do terminal e do Docker imprimem detalhadamente o índice, as transações e o progresso do nonce.

Conclusão

O meu projeto apresenta uma arquitetura que prova ser possível contruir um ecossistema de três pilares: backend seguro e funcional, onde o motor do blockchain foi desenvolvido para garantir a segurança da memória, e também ser capaz de gerir rotas de REST e concorrência, mantendo a rede fluida durante

processo complicados; criptografia e consenso, onde a integração do sha combina com as assinaturas digitais, garantindo a imutabilidade e impedindo falsificações; Infraestrutura p2p virtualizada e uma interface dinâmica, onde a simulação da rede através de contentores isolados em portas distintas funcionou com êxito. Além disso, a interface provou-se capaz de ser intuitiva ao moldar-se ao perfil do utilizador, oferecendo uma experiência visual da auditoria em tempo real e dos logs de mineração.

Referências

Susnjara, S., & Smalley, I. (2025). *What is Blockchain?* IBM.

<https://www.ibm.com/think/topics/blockchain>. Acedido em 26/5/2026

Cavalcanti, L. (2024, December 19). *Blockchain em procurement: vantagens + 6 formas de uso!* Linkana. <https://www.linkana.com/blog/blockchain-em-procurement/>.

Acedido em 26/5/2026

DOS, C. **linguagem de programação**. Disponível em:

<[https://pt.wikipedia.org/wiki/Rust_\(linguagem_de_programa%C3%A7%C3%A3o\)](https://pt.wikipedia.org/wiki/Rust_(linguagem_de_programa%C3%A7%C3%A3o))

>. Acedido em 26/5/2026

axum - Rust. Disponível em: <<https://docs.rs/axum/latest/axum/>>. Acedido em 26/5/2026

Tokio - An asynchronous Rust runtime. Disponível em: <<https://tokio.rs/>>.

Acedido em 26/5/2026

WAGNER, L. **What is SHA-256?** Disponível em:

<<https://www.boot.dev/blog/computer-science/how-sha-2-works-step-by-step-sha-256/>>. Acedido em 26/5/2026

Understanding Ed25519. Disponível em:

<<https://messari.io/copilot/share/understanding-ed25519-ee5c6e19-3f05-4557-ac8f-9a1e84b9e8ff>>. Acedido em 26/5/2026